

第 1 章：强化学习基本设定、MDP（Markov Decision Process，马尔可夫决策过程）与 Bellman 方程

1.1 本章符号说明

本章第一次出现公式前，先把核心符号列清楚：

本章统一写大写 $V_{\pi}(s)$ 和 $Q_{\pi}(s,a)$ ；如果你在 Sutton & Barto 等教材里看到小写 $v_{\pi}(s)$ 、 $q_{\pi}(s,a)$ ，它们表示同类 state-value/action-value function。

符号	English	中文	含义
S / S_t	State / State at time t	状态空间 / 时刻 t 的状态	S 是状态空间， S_t 是时刻 t 的状态随机变量。
s	State realization	具体状态	小写 s 表示某一个具体 state，不是 value function。
A / A_t	Action / Action at time t	动作空间 / 时刻 t 的动作	A 是动作空间， A_t 是时刻 t 的动作随机变量。
a	Action realization	具体动作	小写 a 表示某一个具体 action，不是 action-value function。
R / R_{t+1}	Reward	奖励	R_{t+1} 是执行 A_t 后得到的即时奖励。
G_t	Return	回报 / 累计折扣奖励	G_t 表示从时刻 t 开始的长期累计收益； G 是常用记号，不建议理解成固定英文缩写。
$P(s' / s,a)$	Transition Probability	状态转移概率	从 s 执行 a 后到 s' 的概率。
$\pi(a / s)$	Policy	策略	在状态 s 选择动作 a 的概率。
$V_{\pi}(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的期望 return。
$Q_{\pi}(s,a)$	Action-value Function	动作价值函数	策略 π 下，在 s 先执行 a 的期望 return； Q 可理解为 action quality。
γ	Discount Factor	折扣因子	控制未来 reward 权重的系数。
α	Learning Rate / Step Size	学习率 / 步长	控制一次 TD 或 Q-learning 更新吸收多少新误差信号，通常满足 $0 < \alpha \leq 1$ 。
RL	Reinforcement Learning	强化学习	本课程主题，学习长期决策策略。
POMDP	Partially Observable Markov Decision Process	部分可观测马尔可夫决策过程	观测不满足完整 Markov property 时的建模。
TD	Temporal Difference	时序差分	后续用 Bellman target 做采样更新的方法。

符号	English	中文	含义
DQN / PPO / A2C / SAC	Deep Q-Network / Proximal Policy Optimization / Advantage Actor-Critic / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 优势 Actor-Critic / 软 Actor-Critic	后续章节会学习的典型 deep RL 算法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	最基础的 Monte Carlo policy gradient 算法名。

1.2 本章目标

本章是强化学习的地基。你需要掌握：

- 强化学习和监督学习的区别。
- agent、environment、state、action、reward、policy 的含义。
- return 和 discount factor。
- MDP 五元组和 Markov property。
- $V(s)$ 、 $Q(s,a)$ 的定义和关系。
- Bellman expectation equation 与 Bellman optimality equation。

1.3 本章主线

本章先把 RL 问题压成一个标准数学对象：MDP。顺序是 agent 和 environment 交互，得到 trajectory；trajectory 产生 return；return 的期望定义出 $V_{\pi}(s)$ 和 $Q_{\pi}(s,a)$ ；value function 的自洽关系就是 Bellman equation。后续所有算法基本都在近似求解或优化这些 Bellman 关系。

1.4 强化学习问题设定

1.4.1 强化学习在学什么

强化学习研究的是：一个 agent 通过和 environment 交互，学习一个 policy，使得长期累计 reward 最大。

一次交互可以写成：

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

含义：

- S_t : State at time t , 时刻 t 的状态。
- A_t : Action at time t , 时刻 t 的动作。
- R_{t+1} : Reward after action, 执行 A_t 后得到的奖励。
- S_{t+1} : Next state, 环境转移到的下一个状态。

监督学习通常学习 $x \rightarrow y$ 的映射，而强化学习学习的是 sequential decision making。当前动作不仅影响当前 reward，也会影响未来会遇到的状态分布。

面试表达：

RL 的核心难点在于 delayed reward 和 sequential decision making。Agent 的动作会影响未来状态分布，因此数据不是固定的 i.i.d. 样本，目标也不是单步预测正确，而是长期累计收益最大。

1.4.2 Return

强化学习优化的是累计回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

也就是：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

这里 G_t 表示 Return，也就是“从时刻 t 开始往后看的累计回报”。 G 是强化学习教材中的常用记号，不建议把它理解成某个固定英文缩写。

γ 是 discount factor，通常在 $[0, 1]$ 。

γ 的作用：

- $\gamma = 0$ 时，agent 只关心眼前 reward。
- γ 接近 1 时，agent 更重视长期 reward。
- 在 infinite horizon 下，折扣可以让 return 更容易有界。
- 工程上， γ 也控制短视和长远之间的权衡。

面试追问：为什么需要 discount factor？

可以回答：

一方面是数学上让无限时域的累计 reward 更容易收敛，另一方面表达了对未来 reward 的偏好衰减或不确定性。 γ 越大越重视长期收益，但训练可能更难、更不稳定。

1.4.3 Policy

Policy 描述 agent 如何根据 state 选择 action。

确定性策略：

$$a = \pi(s)$$

随机策略：

$$\pi(a | s) = P(A_t = a | S_t = s)$$

强化学习最终通常是在寻找最优策略：

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[G_t]$$

常见算法分类：

- value-based：先学习价值函数，再通过价值函数导出策略，例如 Q-learning、DQN。
- policy-based：直接参数化策略并优化，例如 REINFORCE、PPO。
- actor-critic：同时学习策略 actor 和价值 critic，例如 A2C、SAC。

1.5 MDP 建模

1.5.1 MDP

MDP 是 Markov Decision Process，强化学习最常见的数学建模。

MDP 五元组：

$$(S, A, P, R, \gamma)$$

分别表示：

- S : State Space, 状态空间。
- A : Action Space, 动作空间。
- $P(s' | s, a)$: Transition Probability, 状态转移概率。
- $R(s, a)$ 或 $R(s, a, s')$: Reward Function, 奖励函数。
- γ : Discount Factor, 折扣因子。

MDP 的核心是假设 Markov property:

$$\begin{aligned} P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots) \\ = P(S_{t+1} | S_t, A_t) \end{aligned}$$

也就是说，当前 state 已经包含了预测未来所需的全部历史信息。

面试表达：

MDP 假设 state 是历史信息的 sufficient statistic。给定当前 state 和 action 后，未来状态转移与更早的历史无关。如果观测不满足这个条件，例如只能看到局部信息，则更准确的模型是 POMDP。

1.6 Value Function 与 Bellman 方程

1.6.1 Value Function

状态价值函数（State-value Function，记作 V ， V 来自 value）：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

含义：从状态 s 出发，之后一直按照策略 π 行动，期望累计 return 是多少。

动作价值函数（Action-value Function，记作 Q ， Q 可理解为 action quality）：

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

含义：在状态 s 先执行动作 a ，之后按照策略 π 行动，期望累计 return 是多少。

两者关系：

$$V_{\pi}(s) = \sum_a \pi(a | s) Q_{\pi}(s, a)$$

最优情况下：

$$V^*(s) = \max_a Q^*(s, a)$$

最优动作：

$$a^* = \arg \max_a Q^*(s, a)$$

为什么 Q 更适合直接决策？

因为 $Q(s,a)$ 已经评价了具体 action 的长期价值，可以直接用 $\arg \max_a Q(s,a)$ 选动作。而 $V(s)$ 只评价状态好坏，如果不知道环境转移模型，单独的 $V(s)$ 不一定能直接告诉你该选哪个动作。

1.6.2 Bellman 方程

Return 可以递归拆开：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

于是价值函数也可以递归表示：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) \mid S_t = s]$$

展开得到 Bellman expectation equation：

$$\begin{aligned} V_{\pi}(s) &= \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \\ &\quad [R(s, a, s') + \gamma V_{\pi}(s')] \end{aligned}$$

对于动作价值函数：

$$\begin{aligned} Q_{\pi}(s, a) &= \sum_{s'} P(s' \mid s, a) \\ &\quad [R(s, a, s') + \gamma \sum_{a'} \pi(a' \mid s') Q_{\pi}(s', a')] \end{aligned}$$

最优 Bellman 方程：

$$\begin{aligned} V^*(s) &= \max_a \sum_{s'} P(s' \mid s, a) \\ &\quad [R(s, a, s') + \gamma V^*(s')] \end{aligned}$$

$$\begin{aligned} Q^*(s, a) &= \sum_{s'} P(s' \mid s, a) \\ &\quad [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')] \end{aligned}$$

这个式子非常重要。Q-learning 和 DQN 都是在用采样和函数逼近来逼近它。

1.6.3 Bellman 推导过程：不要直接背公式

Bellman 方程可以按四步讲。

第一步，从 return 的定义出发：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

把第一项单独拿出来，后面的部分正好是从 $t+1$ 开始的 return：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

第二步，对“从状态 s 出发”的长期回报取期望：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

把递归式代进去：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

第三步，把 G_{t+1} 换成下一状态的价值。因为到达 S_{t+1} 后，后续仍按策略 π 行动：

$$\mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s'] = V_{\pi}(s')$$

于是得到：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

第四步，把期望展开成“先按策略选动作，再按环境转移到下一个状态”：

$$V_{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_{\pi}(s')]$$

这就是 Bellman expectation equation。

从 expectation 到 optimality 只改一个地方：不再对动作按当前策略求平均，而是假设在每个状态都选最优动作：

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')]$$

面试表达：

Bellman 方程本质是价值函数的递归自洽关系。当前状态的价值等于即时 reward 加上下一个状态价值的折扣期望。Expectation version 是评估固定策略，optimality version 是每一步都对动作取最大值。

1.6.4 从 Bellman 到算法的推理链

不同 RL 算法可以看成在回答同一个问题：怎么逼近 Bellman backup？

方法	推理方式	需要环境模型吗	用什么近似 Bellman target
DP	直接对 $P(s' s, a)$ 求和	需要	完整期望 backup
MC	采样完整 episode	不需要	实际 return G_t
TD	采样一步 transition	不需要	$r + \gamma V(s')$
Q-learning	采样一步 transition，并对下一动作取 max	不需要	$r + \gamma \max_a Q(s', a')$
DQN	用神经网络拟合 Q-learning target	不需要	$r + \gamma \max_a Q(s', a'; \theta)$

所以不要把这些算法看成完全无关的技巧。它们都围绕 Bellman target，只是期望怎么估、target 怎么构造、是否用函数逼近不同。

1.6.5 TD Target 与 TD Error

Q-learning 的更新式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

其中：

$$TD\ target = r + \gamma \max_{a'} Q(s', a')$$

$$TD\ error = TD\ target - Q(s, a)$$

直觉：

- 旧估计是 $Q(s, a)$ 。
- 新观测告诉我们，一步之后的目标应该是 $r + \gamma \max_{a'} Q(s', a')$ 。
- 两者差值就是 TD error。
- 用学习率 α 做一个增量更新。

1.7 本章例子：把问题建模成 MDP

1.7.1 例子 1：GridWorld 迷宫

一个机器人在网格里走，从起点走到终点。

MDP 元素	对应例子
State	机器人当前格子坐标，例如 (2,3)。
Action	上、下、左、右。
Transition	执行动作后到达相邻格子，也可能因为墙或噪声停在原地。
Reward	到终点 +10，撞墙 -1，每走一步 -0.1。
Policy	在每个格子选择哪个方向走。
Value	从某个格子出发，长期来看能拿到多少 reward。

这个例子的价值在于直观：离终点近、不容易撞墙的状态价值高；容易掉进惩罚区域的状态价值低。

1.7.2 例子 2：推荐系统

用户打开 App，系统要选择展示什么内容。

MDP 元素	对应例子
State	用户画像、最近点击、当前场景、时间、设备。
Action	推荐某个视频、商品或文章。
Reward	点击、停留、购买、负反馈等综合信号。
Transition	用户看完内容后兴趣状态变化。
Policy	给不同用户和场景选择不同推荐内容。

如果只看单次点击，这更像 bandit；如果考虑长期留存、疲劳、兴趣迁移，就更像完整 RL。

面试表达：

我会先问这个问题有没有长期影响。如果动作只影响当前 reward，可以先用 bandit；如果动作会改变后续状态，例如用户兴趣或库存状态，就需要 MDP/RL 视角。

1.8 本章面试高频问题

1.8.1 RL 和监督学习有什么区别？

监督学习有固定标签，通常假设样本 i.i.d.，目标是拟合输入到标签的映射。RL 没有逐步标准答案，只有 reward，且 reward 可能延迟出现。Agent 的动作会改变未来状态分布，因此需要处理 exploration-exploitation 和长期收益最大化。

1.8.2 什么是 MDP？

MDP 是强化学习的标准建模，由状态空间、动作空间、转移概率、奖励函数和折扣因子组成。它的核心假设是 Markov property，即未来只依赖当前状态和动作，而不依赖完整历史。

1.8.3 V 和 Q 有什么区别？

$V_\pi(s)$ 衡量状态 s 在策略 π 下的长期价值。 $Q_\pi(s,a)$ 衡量在状态 s 先执行动作 a 后，再跟随策略 π 的长期价值。 Q 多了动作维度，因此更适合直接用来选动作。

1.8.4 Bellman expectation equation 和 Bellman optimality equation 区别？

Bellman expectation equation 用来评估一个给定策略 π 的价值。Bellman optimality equation 描述最优价值函数，会在动作上取最大值，对应寻找最优策略。

1.8.5 Bellman 方程的核心直觉是什么？

当前价值等于即时奖励加上折扣后的未来价值。它把长期决策问题拆成一步 reward 和下一状态 value 的递归组合。

1.9 本章练习

1. 用自己的话解释为什么强化学习的数据不是固定 i.i.d.。
2. 手写 $V_\pi(s)$ 、 $Q_\pi(s,a)$ 的定义，以及 $V_\pi(s)$ 和 $Q_\pi(s,a)$ 的关系。
3. 从 $G_t = R_{t+1} + \gamma G_{t+1}$ 推导 Bellman expectation equation。
4. 解释为什么 $Q^*(s,a)$ 的 Bellman optimality equation 中有 $\max_a$ 。

▼ 参考答案（点击展开）

1. 为什么 RL 数据不是固定 i.i.d.：相邻 transition 来自同一条 trajectory，时间上有相关性；当前 policy 决定会访问哪些 state 和 action，而 policy 又会随训练更新；agent 的 action 还会改变后续 state。因此数据分布会被策略和交互过程共同改变，不是预先固定、相互独立且同分布的样本集。
2. V 、 Q 的定义和关系： $V_\pi(s) = \mathbb{E}_\pi[G_t | S_t=s]$ ； $Q_\pi(s,a) = \mathbb{E}_\pi[G_t | S_t=s, A_t=a]$ 。二者满足 $V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s,a)$ ；反过来， $Q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t=s, A_t=a]$ 。
3. Bellman expectation 推导：从 $G_t = R_{t+1} + \gamma G_{t+1}$ 出发，对条件 $S_t=s$ 取期望： $V_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t=s]$ 。根据下一状态值的定义，用 $V_\pi(S_{t+1})$ 替换对 G_{t+1} 的条件期望，得到 $V_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t=s]$ 。
4. 为什么有 $\max_a$ ： $Q(s,a)$ 假设当前先执行 a ，从下一状态开始继续采用最优策略。因此到达 s' 后应选择价值最大的下一动作，即 $\max_{a'} Q(s',a')$ ；它表达的是“最优延续”，不是环境随机性。